



分支限界法

—An Efficient Searching Algorithm

朱容波

Email: rongbozhu@126.com

Office: 一综B323

本章内容



- Branch and bound分支限界法
 - 15谜问题
 - TSP 旅行商问题

分支界限法

- 回溯法和分支限界法都是通过系统地搜索解空间树而求得问题解的搜索算法。

在系统地检查解空间的过程中，抛弃那些不可能导致合法解的候选解，从而使求解时间大大缩短。（与蛮干算法相区别）

- 回溯法以深度优先的方式搜索解空间，而分支限界法以广度优先或最小耗费（最大收益）的方式搜索解空间。

分支限界法与回溯法对比

	回溯法	分治限界法
对解空间的搜索方式	深度优先搜索	广度优先或最小消耗优先搜索
存储结点的常用数据结构	堆栈	队列、优先队列
结点存储特征	活结点的所有可行子结点被遍历后才被从栈中弹出	每个结点只有一次成为活结点的机会
常用应用	找出满足约束条件的所有解	找出满足约束条件的一个解

13.5 Branch and Bound(分支限界法)

- **分支-限界法**：在生成当前E-结点的全部儿子之后再生成其它活结点的儿子，并且，用限界函数帮助避免生成不包含答案结点的子树。
- 检索策略分为：
 - **FIFO(first in first out)检索**：类似于BFS的状态空间检索，它的活结点表采用一张先进先出表(队列)
 - **LC-检索（最小耗费检索）**

队列式（FIFO）搜索策略的缺陷

- 队列式（FIFO）**搜索策略的缺点**：对下一个E-结点的选择相当死板,而且在某种意义上是盲目的。这种选择对于有可能快速检索到一个答案结点的结点**没有任何优先权**。
- 队列式（FIFO）分支限界法其采用的是广度优先搜索方法，虽然可以保证解的深度最少，但不能保证搜索朝着最佳解的方向进行下去。

基于最小代价 (Least-cost, LC) 搜索策略的分支限界法

- 解决: 对活结点使用一个“有智力的”**排序函数 $c(.)$** 来选取下一个E-结点, 使有可能产生答案结点的活结点优先称为E-结点, 从而可以加快到达一个答案结点的检索速度。
- 此时若适当地基于最小代价 (LC) 的搜索策略, 则可以通过 $c(n)=d(n)+w(n)$ 来近似估计取得最优解的可能性, 从而尽可能的减少中间状态.

找最小成本答案结点的分枝-限界算法

- 使用 $\hat{c}(X) \leq c(X)$ 的成本估计函数 $\hat{c}(\cdot)$ 来给出由任一结点X得出的解的下界，使算法具有一定的智能，减少盲目性；
- 定义U是最小成本解的成本上界， 则：
 - (1) 具有 $\hat{c}(X) > U$ 的活结点X可以被杀死，这是因为由X可以到达的答案结点X有 $c(X) \geq \hat{c}(X) > U$ ；
 - (2) 如果已经到达一个具有成本U的答案结点，则 $\hat{c}(X) \geq U$ 的所有活结点都可以被杀死
 - (3) U的初始值可以用某种启发式方法得到，也可以置成 ∞

15-谜问题(例子)

◆ 在一个分成 4×4 的棋盘上排列15块号牌，其中会出现一个空格。棋盘上号牌的一次合法移动是指将位于空格上、下、左、右的一块号牌移入空格。要求通过一系列合法移动，将号牌的初始排列转换成自然排列。

◆ 例

1	3	4	15
2		5	12
7	6	11	14
8	9	10	13

(a) 初始状态 1

1	2	3	4
5	6		8
9	10	7	11
13	14	15	12

(b) 初始状态 2

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

(c) 目标状态

15-谜问题(规模分析)

- ▶ 本题中空格移动是有限制的。棋盘上号牌的一次合法移动是指将于空格上、下、左、右的一块号牌移入空格。要求将号牌的初始排列转换为自然排列。
- ▶ 通过分析可知共有 $16!$ 种不同的状态，有且仅有一个状态为目标状态。这个状态空间树是相当大的。因此判定当前到达的状态是否属于该状态空间树是非常有必要的。
- ▶ 值得注意的是：有定理指出并不是任意一个初始状态都可经过有限步的移动来到达目的状态。

可行性分析:

1. 棋盘上方格编号

如果将棋盘上的各个位置按照号牌的自然排列发个序号，右下角为16号。

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

可行性分析:

2. 记**POSITION(i)**为编号*i*的牌在**初始状态**中的位置;
POSITION(16)表示空格的位置。
3. 初始状态1和状态2的POSITION图如下图POSITION_1和POSITION_2所示。

1	3	4	15
2		5	12
7	6	11	14
8	9	10	13

(a) 初始状态 1

1	5	2	3
7	10	9	13
14	15	11	8
16	12	4	6

图POSITION_1

1	2	3	4
5	6		8
9	10	7	11
13	14	15	12

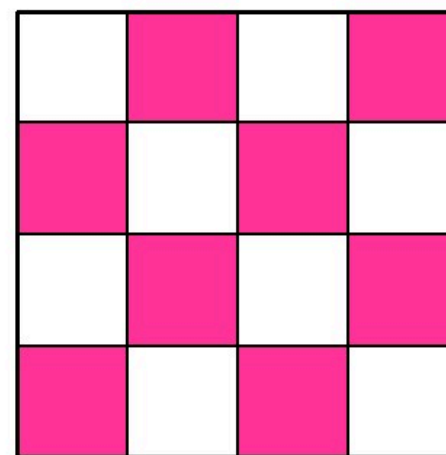
(b) 初始状态 2

1	2	3	4
5	6	11	8
9	10	12	16
13	14	15	7

图POSITION_2

可行性分析:

4. 记LESS(i)是这样牌j的数目:
 $j < i$, 但 $POSITION(j) > POSITION(i)$,
即编号小于i但初始位置i之后的牌的数目。
5. 引入一个量X
如图所示, 初始状态时,
若空格落在红色方格上, 则 $X=1$;
若空格落在白色方格上, 则 $X=0$;



可行性分析:

6. 初始状态能否经过若干步合法移动到达目标状态,
可由定理1给出:

定理1 当且仅当 $\sum_{i=1}^{16} LESS(i) + X$ 是偶数时, 目标状态才能由初始状态经过若干步后到达;

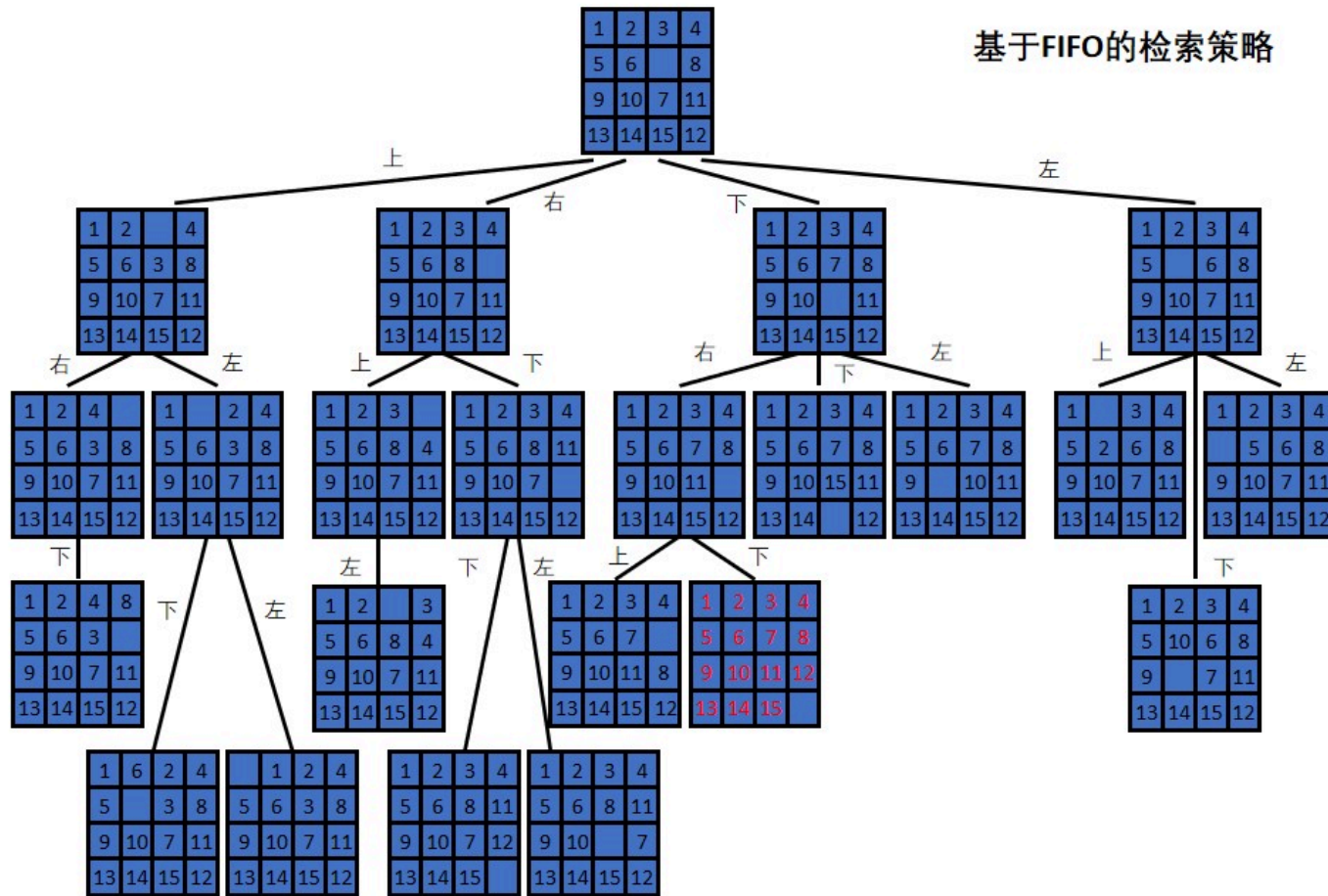
7. 证明示例:

对于初始状态2而言, 值为 $15+1=16$;

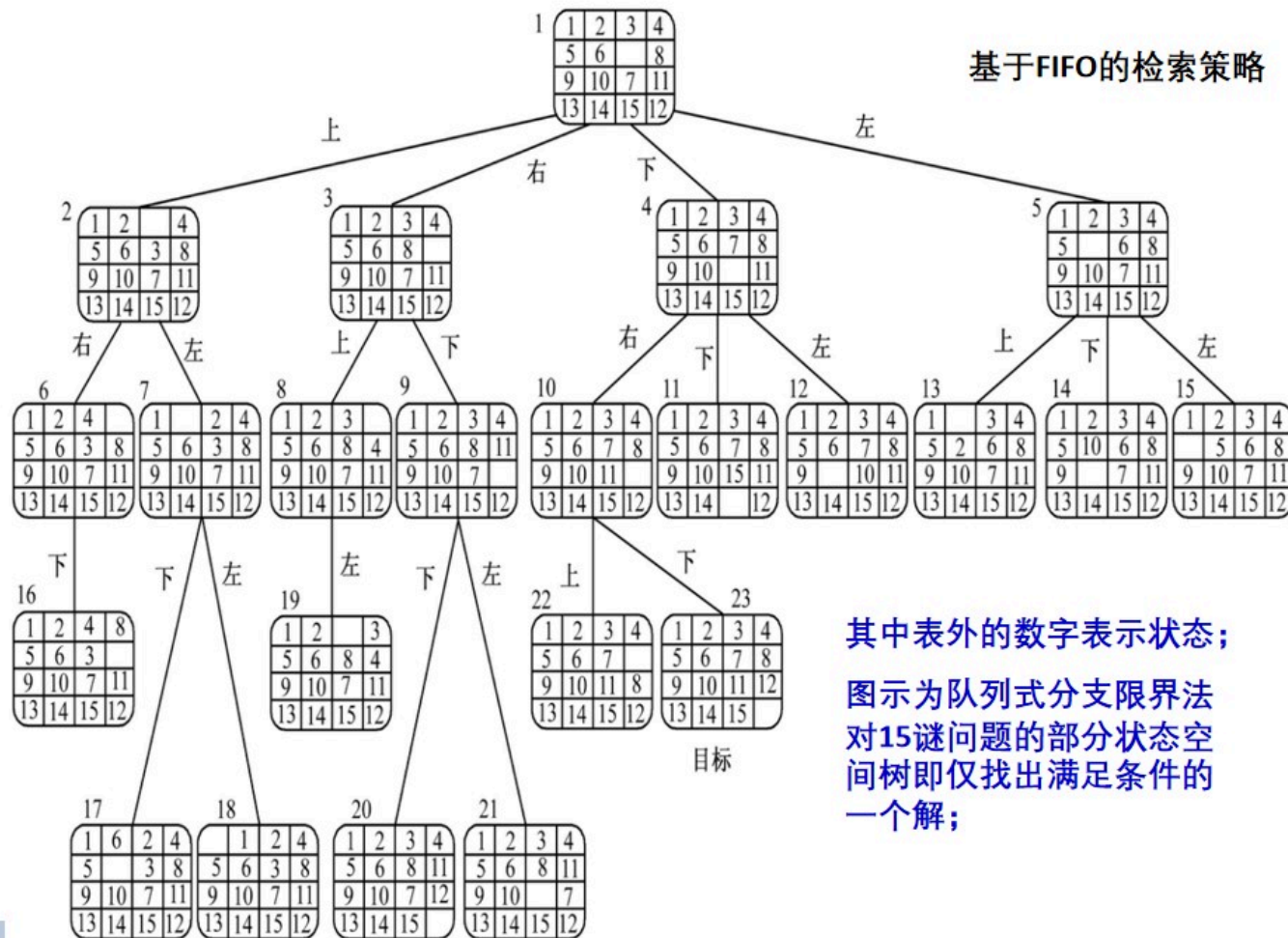
8. 结论: 初始状态2可转换为自然排列;

下面看如何通过FIFO的搜索策略来求解15迷问题

基于FIFO的检索策略



基于FIFO的检索策略

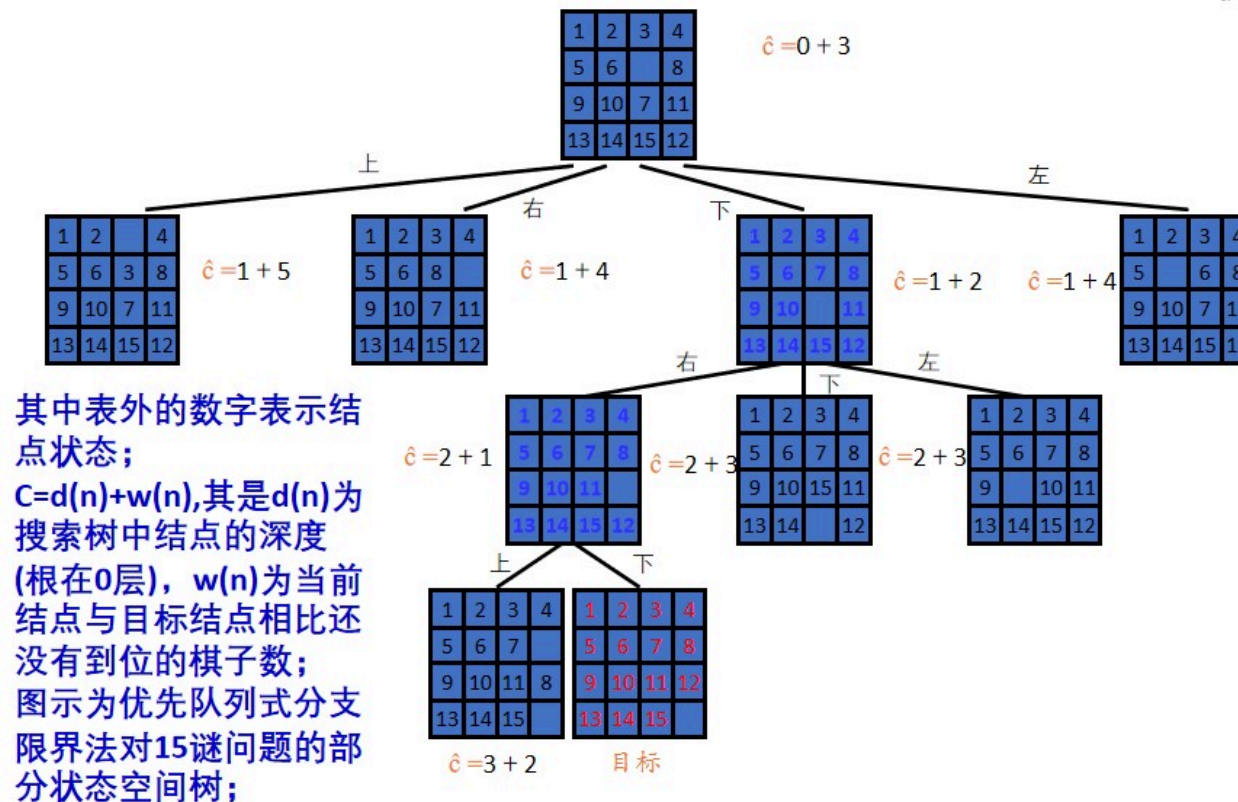


下面看如何通过基于最小代价（LC）搜索策略的分支限界法来求解15迷问题

基于最小代价 (Least-cost, LC) 搜索策略的分支限界法

- 成本函数 $c(.)$: 从根出发到最近目标结点路径上的每个结点赋予这条路径的长度作为它们的成本。
- 估计**成本函数** $\hat{c}(.)$: $\hat{c}(X)=h(X)+\hat{g}(X)$,
 - $h(X)$ 是由根到结点 X 的路径的长度;
 - $\hat{g}(X)$ 是对以 X 为根的子树中由 X 到目标状态的一条路径长度的估计。它至少应是能把状态 X 转换成目标状态所需的最小移动数。
 $\hat{g}(X)=$ 不在其目标位置的非空白牌数目
- 可以看出 $\hat{c}(X)$ 是 $c(X)$ 的下界。

最小代价检索



最小代价搜索策略的一般过程

- LC-检索与FIFO, LIFO的区别在于得到下一个E-结点的规则不同.
- $c(\cdot)$: 状态空间树T中结点的成本函数; $c(X)$ 是其根为X的子树中任一答案结点的最小成本; $c(T)$ 是T中最小成本的答案结点的成本;
- 通常不可能找到如上定义的易于计算的 $c(\cdot)$, 故用一个对 $c(\cdot)$ 估值的启发式函数 $\hat{c}(\cdot)$ 代替.
- $\hat{c}(\cdot)$ 一般易于计算且有如下性质: 如果X是一个答案结点或是一个叶子结点, 则有 $c(X) = \hat{c}(X)$

Algorithm LC-SEARCH

Input: 状态空间树T, T中结点的估计成本函数 $\hat{C}$ 。

Output: 问题的解 (从根到答案结点的路径)

1. if T 是答案节点 then 输出T and return; end if
2. $E \leftarrow T$ //E-结点
3. 将活结点表初始为空
4. repeat
5. for E 的每个儿子X
6. if X 是答案结点 then
7. 输出从X到T的路径, return
8. end if
9. call ADD(X) //将新的活结点X加到活结点表中
10. PARENT(X) $\leftarrow$ E //E结点标记为X的父结点
11. end for
12. if 活结点表为空 then
13. 输出“无解”, return
14. end if
15. call LEAST(E) //找一个具有最小 $\hat{C}$ 值的活结点放到E中, 从活结点表中删除
16. end repeat

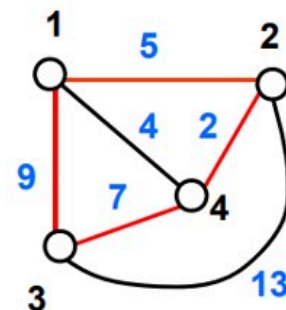
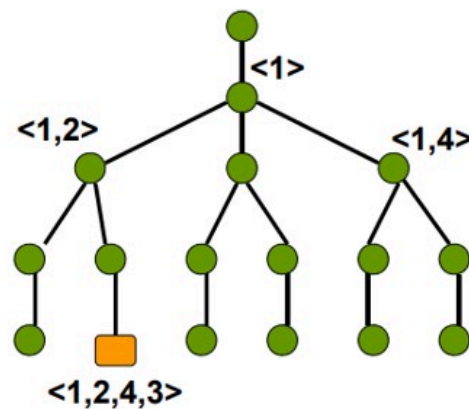
9.5 TSP Problem (旅行商问题)

? 问题描述

- 给出一个城市的集合和一个定义在每一对城市间的耗费函数，找出耗费最小的旅行。在这里，一个旅行是一个闭合的路径，它访问每个城市恰好一次，也就是说，是一个简单回路。耗费函数可以是距离、旅行时间、飞机票价等。

TSP Problem (旅行商问题)

$\langle i_1, i_2, \dots, i_n \rangle$ 为巡回路线
搜索空间：排列树, $(n-1)!$ 片树叶
实例：



$\langle 1, 2, 4, 3 \rangle$ 对应于巡回路线: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

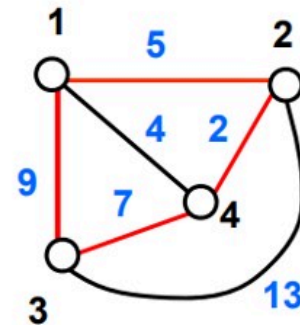
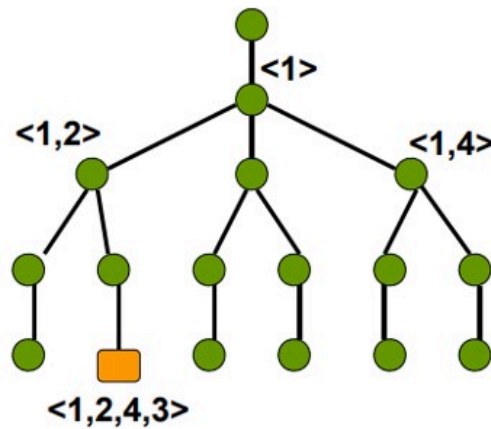
长度: $5 + 2 + 7 + 9 = 23$

TSP Problem (旅行商问题)

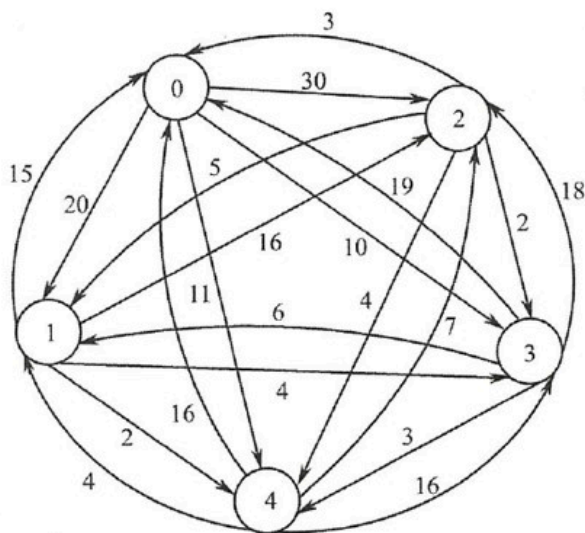
- 一个可行解是 n 个城市的一个排列，排列的第一个元素是旅行商的出发地。最后一个城市是其周游的最后一个城市，由此城市又回到出发地
- 如果城市编号为 $1, 2, \dots, n$ ，其出发地城市是 1 ，则一个可行解即为首元素是 1 的一个 n 元排列

TSP Problem (旅行商问题)

- 这些排列可以安排成如下的搜索树，从根节点到叶节点的一条路径对应了 $\{1,2,\dots,n\}$ 的排列



TSP Problem (旅行商问题)



- 矩阵约减过程可理解为对原有向图进行某种处理，使得边上的权值变小，但图的结构不变

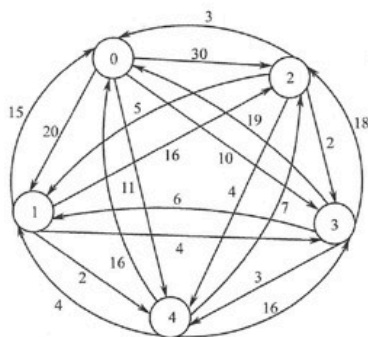
$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

(a) 代价矩阵

TSP Problem (旅行商问题)

- 归约代价矩阵

- 对第*i*行规约是将该行的每个元素减去该行的约数 t_i 得到新行。
- 对矩阵的一行进行归约，可以将该行中的每个元素减去该行的最小数进行，此最小数称为该行的约数。



$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$	$\begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$
--	--

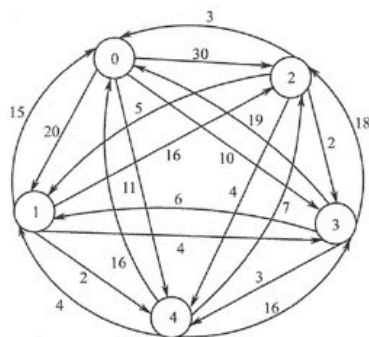
(a) 代价矩阵

(b) 逐行归约

TSP Problem (旅行商问题)

- 归约代价矩阵

- 同理，**规约第j列**等价于对以顶点j为头的所有边的权值都减去该列的约数 r_j ，得到一个与原图结构相同的有向图，但每条周游路径长度将因此减少 r_j 。
- 对矩阵的**一列**进行归约，可以将该**列**中的每个元素减去



$$\begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$$

(b) 逐行归约

$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

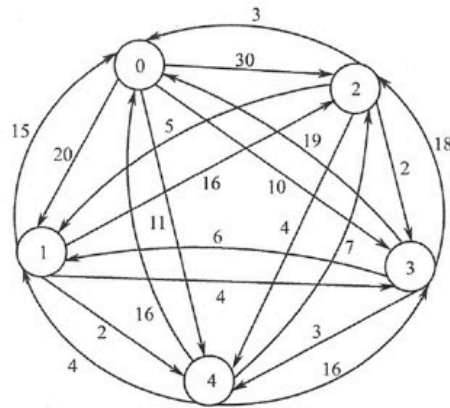
(c) 逐列归约

矩阵约数 $L=(10+2+2+3+4)+(1+0+3+0+0)=25$

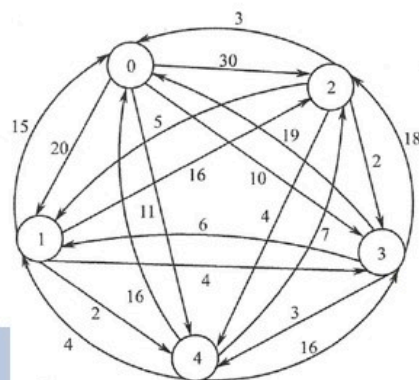
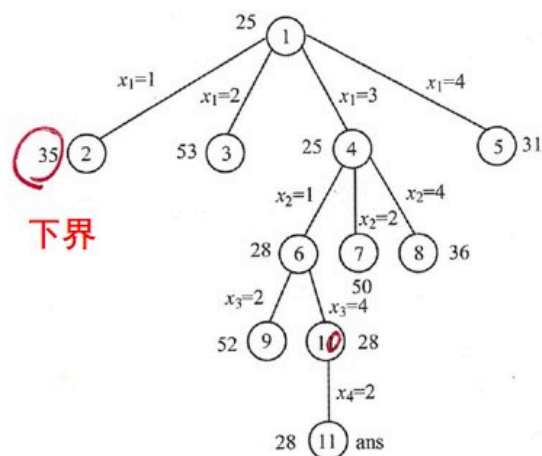
下面看如何通过基于最小代价（LC）搜索策略的分支限界法来求解TSP旅行商问题

基于LC分支限界的TSP Problem（旅行商问题）

- 根的下界函数：由于归约代价矩阵是非负的，这意味着由归约代价矩阵所代表的周游路径长度仍有非负值，所以矩阵约数L的长度不大于原图的任意一条周游路径长度，可将L作为旅行商问题状态空间树根R的下界函数值，即 $\hat{c}(R) = L$ ，必有 $\hat{c}(R) \leq c(R)$ ， $c(R)$ 是图

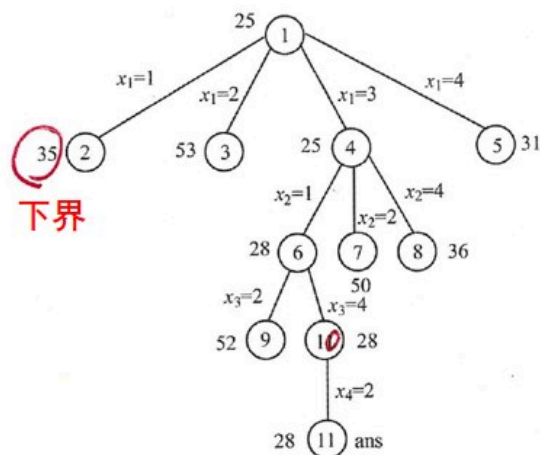


基于LC分支限界的TSP Problem (旅行商问题)

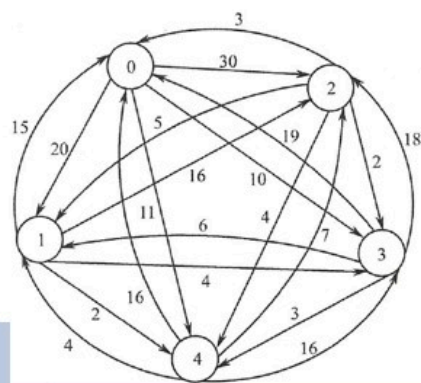


- (1) 根结点R的归约矩阵是对邻接矩阵直接归约得到的，其下界函数 $\hat{c}(R) = L$ ，L是矩阵约数。
- 因为原图邻接矩阵的矩阵约数 $L=(10+2+2+3+4)+(1+0+3+0+0)=25$ ，所以状态树根结点的下界函数值是 $\hat{c}(1) = 25$

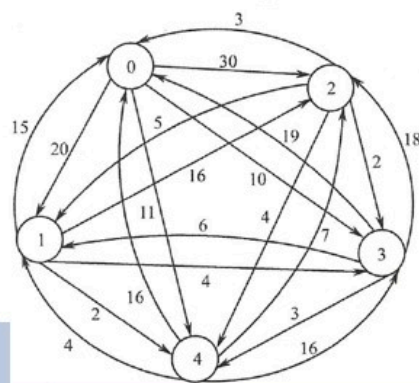
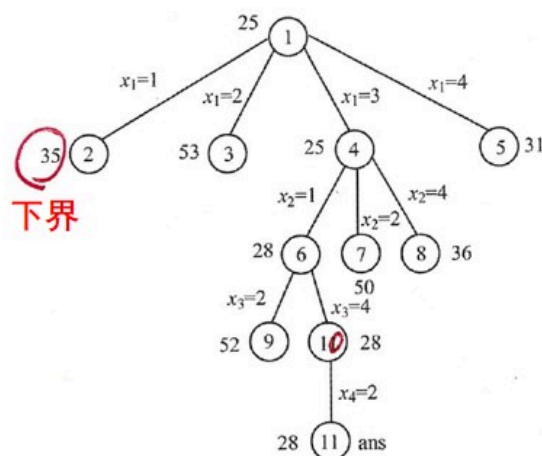
基于LC分支限界的TSP Problem (旅行商问题)



- (2) 状态树中任意非叶状态X的归约矩阵B, 可由其双亲状态P的归约矩阵A先变换成A'后, 再归约得到B; X的下界函数 $\hat{c}(X) = \hat{c}(P) + A[i][j] + r$, r是从A'归约到B的矩阵约数



基于LC分支限界的TSP Problem (旅行商问题)



- (接上页) 例: 图 (a) 是结点1的归约矩阵。结点1是结点3的双亲结点, 结点1到结点3的边代表有向图的边 $\langle 0,2 \rangle$
- 为了计算结点3的归约矩阵和下界函数值, 首先需要将结点1的归约矩阵的第0行、第2列和元素 $a[2][0]$ 都置为 ∞ , 得到 (b) 矩阵

$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

(a) 结点1的归约矩阵

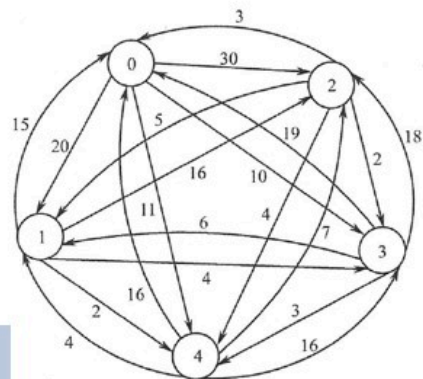
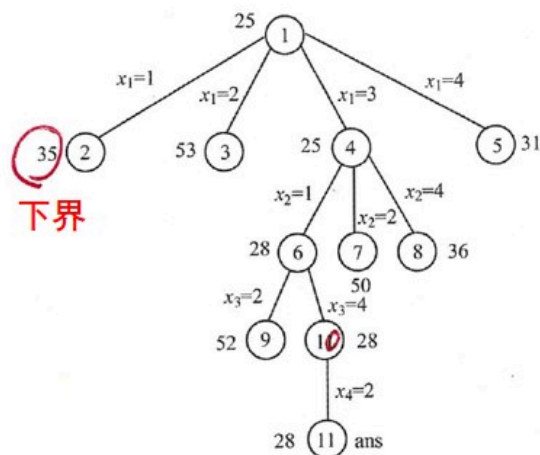
$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 & 3 & \infty & \infty & 0 \\ 11 & 0 & \infty & 12 & \infty \end{bmatrix}$$

(b) 置 ∞ 操作

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 4 & 3 & \infty & \infty & 0 \\ 0 & 0 & \infty & 12 & \infty \end{bmatrix}$$

(c) 结点3的归约矩阵

基于LC分支限界的TSP Problem (旅行商问题)



- (接上页) 然后在对图(b)矩阵进行归约, 得到图 (c) 矩阵, 即结点3的归约矩阵. 由于此时边 $\langle 0,2 \rangle$ 的权为17, 本次归约的矩阵约数为11, 且 $\hat{c}(1)=25$, 因此, $\hat{c}(3)=25+17+11=53$

$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

(a) 结点 1 的归约矩阵

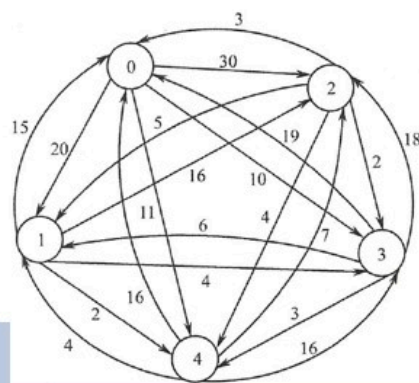
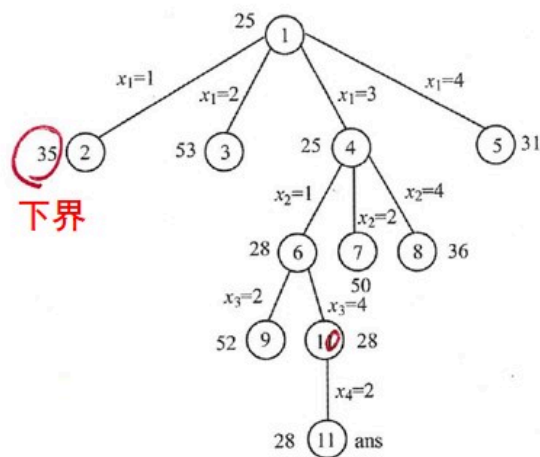
$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 & 3 & \infty & \infty & 0 \\ 11 & 0 & \infty & 12 & \infty \end{bmatrix}$$

(b) 置 ∞ 操作

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 4 & 3 & \infty & \infty & 0 \\ 0 & 0 & \infty & 12 & \infty \end{bmatrix}$$

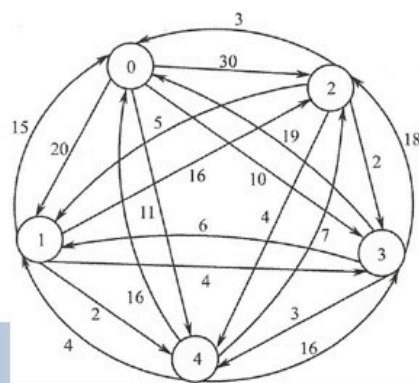
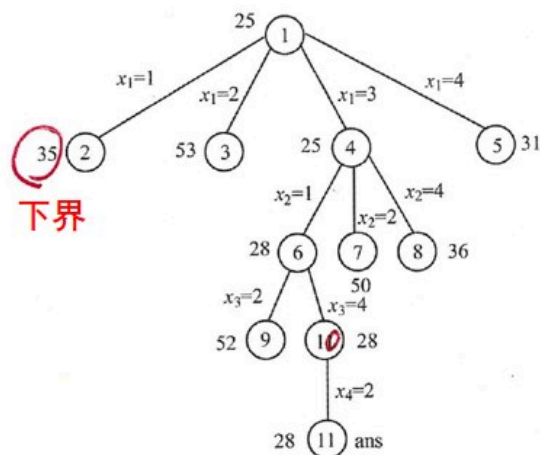
(c) 结点 3 的归约矩阵

基于LC分支限界的TSP Problem (旅行商问题)



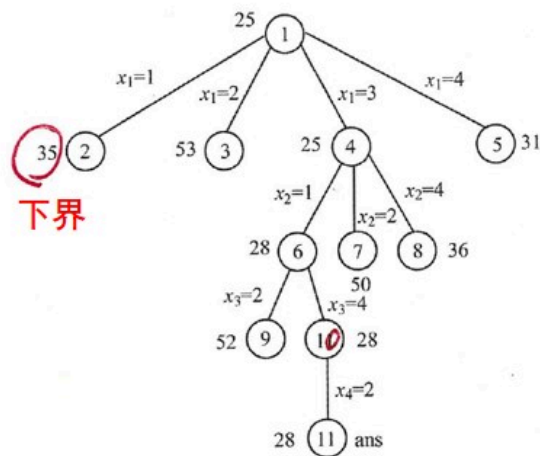
- 对于树中任何状态结点 X , 令上界函数值 $u(X) = \infty$
- 算法首先生成状态空间树的根结点1, 并有 $U = \infty$
- 接着生成结点1的所有孩子结点2、3、4和5, 依次进优先权队列, 他们的下界函数为优先权。
- 结点4最先出队列成为E-结点, 算法接着生成结点6,7和8

基于LC分支限界的TSP Problem (旅行商问题)

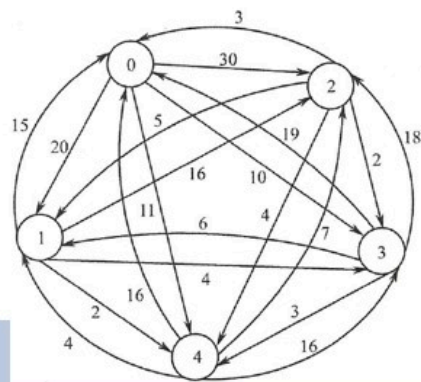


- (接上页) 下一个出队列的结点为结点6, 并生成结点6的孩子9和10.
- 随后结点10出队列, 生成结点11. 结点11是答案结点
- 根据LC分支限界算法, 将修改 $U=28$, 下一个从优先权队列输出的结点是结点5, 其下界函数值为31, 此时已有 $\hat{c}(5) > U$, 即 $31 > 28$, 算法结束。

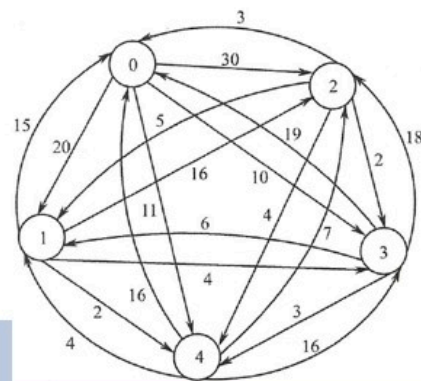
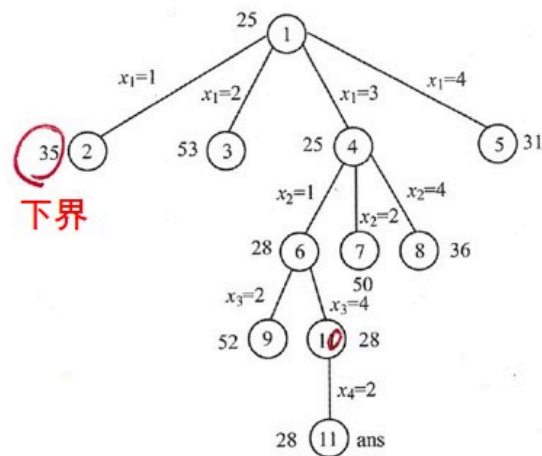
基于LC分支限界的TSP Problem (旅行商问题)



- (接上页) 得到的最小代价周游路径为(0,3,1,4,2,0), 该路径长度为28



基于LC分支限界的TSP Problem (旅行商问题)



- 下界函数的计算从图的邻接矩阵开始，使用矩阵归约和变换，得到旅行商问题的状态空间树上所有的状态结点的归约矩阵和下界函数值